



Recovery



Content

- **Introduction**
 - Failure modes
 - Purpose of recovery
- Transaction log
- Checkpoint
- Recovery method

Remind

- Data integrity
 - Correctness of the data
 - Consistency of the constraints
- Consistent state
 - All integrity constraints are satisfied
- Consistent database state
 - The database remains in a consistent state

Introduction

- How are constraints violated?
 - Transaction bug
 - Programming bug of DBMS
 - Hardware failure
 - Data sharing
- How to fix and restore the data?
 - Data sharing : last lectures
 - Failure : this lecture

Failure mode



- Erroneous data entry
- Media failure
- Transaction failure
- System failure

Erroneous data entry

- The data is evidently in error
 - Omits a digit from your phone number
 - Do not have the form of a phone number
- Some data errors are impossible to detect
 - Mistypes one digit of the phone number
- DBMS provides mechanisms for catching data-entry errors that are detected
 - Key constraints, foreign key constraints
 - Constraints on values
 - Triggers

Media failure

- A major failure of a disk where the entire disk becomes unreadable
 - Lost the data on the disk
 - Cannot access the disk
 - Sectors are damaged
 - The reader is damaged
- DBMS handles by
 - RAID schemes
 - Maintain a copy of the database on a medium
 - Tape, optical disk
 - Keep redundant copies of the database online, distributed among several sites

Media failure

■ Note

- Catastrophic failure
 - The media holding the database is completely destroyed
- Eg
 - Explosions
 - Fires
 - Vandalism at the site of the database
- RAID will not help
- Archiving and redundant, distributed copies can be used to protect against catastrophic failure

Transaction failure

- A failure that makes a transaction to ends unusually
 - Divide by zero
 - Transactions are aborted
 - Erroneous data entry
 - Arithmetic overflow

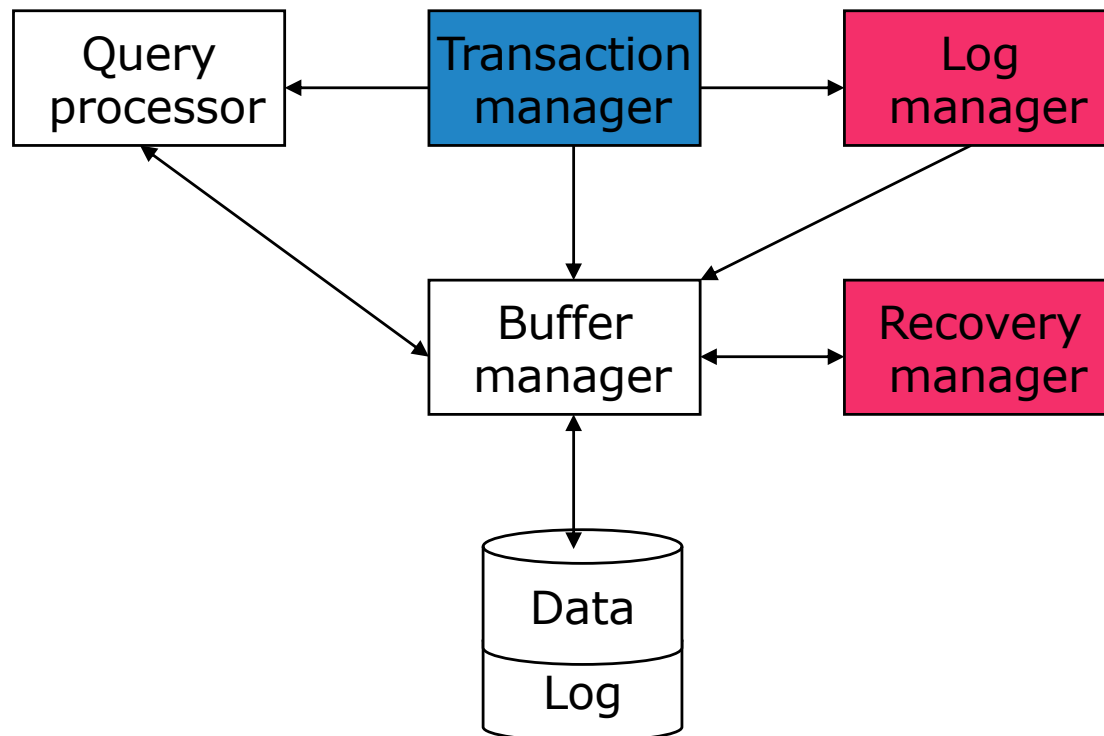
- DBMS
 - Redo the transaction

System failure

- Problems that cause the state of a transaction to be lost
 - Power lost
 - Cause the contents of main memory to disappear (transactions occur in main memory)
 - Software errors
 - Overwrite part of main memory
- DBMS
 - Transaction log

Purpose of recovery

- Put database in the last consistent state before failure
- Assure that the transaction are
 - Atomic
 - Durability



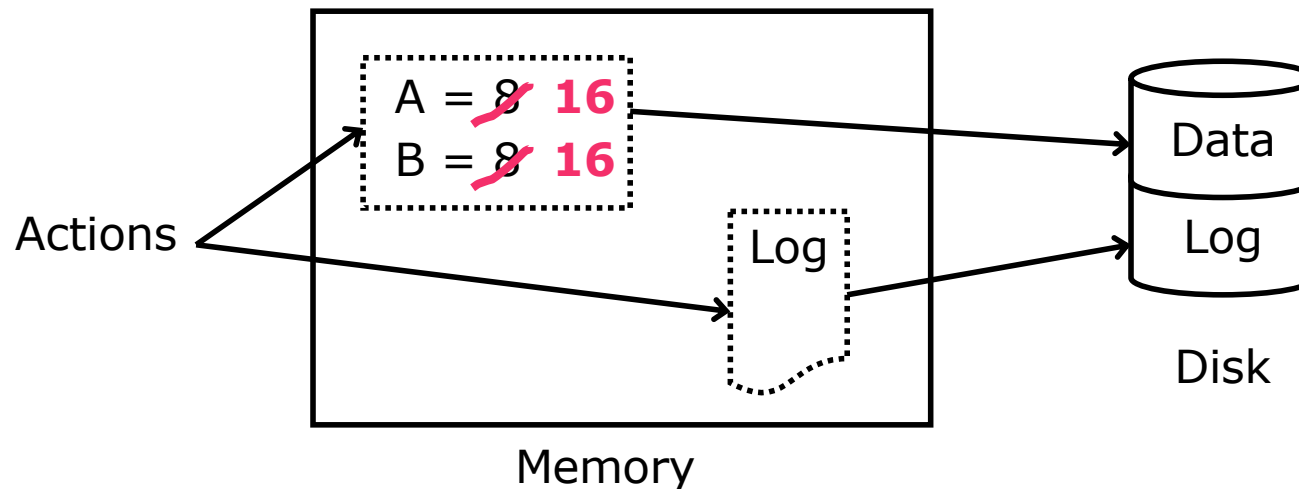
Content

- Introduction
- **Transaction log**
- Checkpoint
- Recovery method

Transaction log

■ A log

- A sequence of **log records**, each telling something about what some transaction has done
- A sequential file is stored in the main memory, and will be written to disk as soon as possible



Transaction log

- A log record

- <start T>
 - transaction T has begun
- <commit T>
 - transaction T has completed successfully and will make no more changes to database element
- <abort T>
 - transaction T could not complete successfully
- <T, X, v> or <T, X, v, w>
 - transaction T has changed database element X, its former value was v and its new value is w

Transaction log

- If there is a system crash
 - The log is consulted to reconstruct what transactions were doing when the crash occurred
- To repair the effect of the crash
 - Some transactions will have their work ***done again (redo)***
 - The new values they wrote into the database are written again
 - Other transactions will have their work ***undone (undo)***
 - The database restored so that it appears that they never executed

Content

- Introduction
- Transaction log
- **Checkpoint**
- Recovery method

Discussion

- Recovery requires the entire log be examined, in principle
 - Scan the log backwards from the end to identify incomplete transactions
 - Once a transaction has its *<Commit T>* log record written to disk
 - No longer needed during recovery
 - take time
 - Could not delete the log prior to a *<Commit T>*
 - Many transactions execute at once
- *Checkpoint* the log periodically

Checkpoint

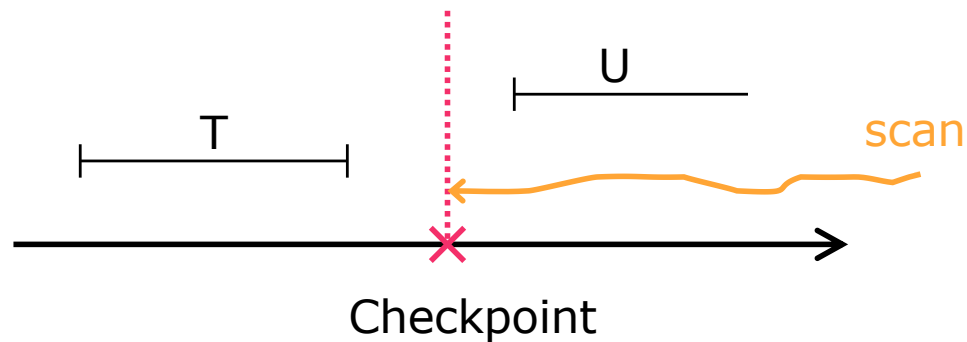
- Transaction log has one more log record
 - **<checkpoint>** or **<ckpt>**
- Log record **<checkpoint>**
 - Will be written periodically
 - At the time when DBMS writes all changes of the database from main memory to disk
- Category
 - Simple checkpoint
 - Nonquiescent checkpoint

Simple checkpoint

- At the time of checkpoint
 - (1) Stop accepting new transactions
 - (2)
 - Wait until all currently active transactions commit or abort
 - Have written <Commit T> or <Abort T> record on the log
 - (3) Flush the log to disk
 - (4)
 - Write a log record <Checkpoint>
 - Flush the log again
 - (5) Resume accepting transactions

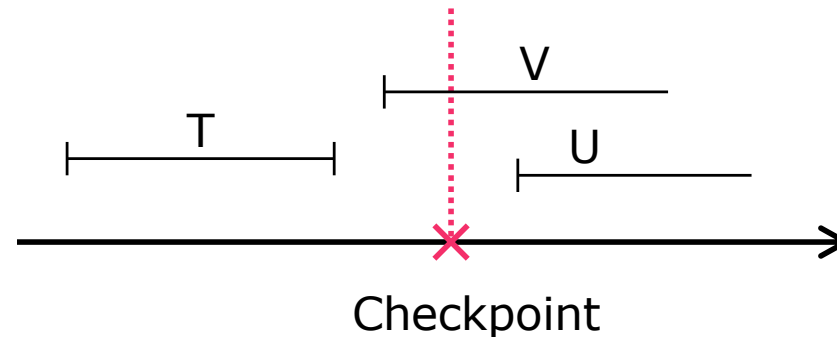
Simple checkpoint

- Any transaction that executed
 - Prior to the checkpoint will have finished
 - No need to undo any of these transactions
 - After to the checkpoint have not finished yet
 - Need to recover
- No need to scan prior to the checkpoint



Nonquiescent checkpoint

- During the checkpoint
 - System nearly stops working
 - Because active transactions may take a long time to commit or abort
- More complex technique
 - Allows new transactions to enter the system during the checkpoint



Nonquiescent checkpoint

- Log record
 - <start checkpoint ($T_1, \dots, T_n$)> or <start ckpt ($T_1, \dots, T_n$)>
 - $T_1, T_2, \dots, T_n$ are active transactions
 - Have not yet committed and written their changes to disk
 - <end checkpoint> or <end ckpt>

Nonquiescent checkpoint

- At the time of checkpoint
 - (1)
 - Write a log record $\langle \text{start ckpt } (T_1, \dots, T_n) \rangle$
 - Flush the log
 - (2)
 - Wait until all of $T_1, \dots, T_n$ commit or abort
 - Do not prohibit other transactions from starting
 - (3)
 - When all of $T_1, \dots, T_n$ have completed, write a log record $\langle \text{end ckpt} \rangle$
 - Flush the log

Nonquiescent checkpoint

- Recovery depends on
 - First meet <end ckpt>
 - All incomplete transactions began after the previous <start ckpt ($T_1, \dots, T_n$)>
 - Scan backwards as far as the next <start ckpt> and then stop
 - First meet <start ckpt ($T_1, \dots, T_n$)>
 - $T_1, \dots, T_n$ did not complete before the crash
 - Need to scan no further back than the start of the earliest of the incomplete transactions

Content

- Introduction
- Transaction log
- Checkpoint
- Recovery method
 - **Undo-logging (immediate modification)**
 - Redo-logging (deferred modification)
 - Undo/Redo logging

Undo-Logging

■ Rules

- (0) Log record
 - $\langle T, X, v \rangle$
 - Only write the old value
- (1) The log record $\langle T, X, v \rangle$ must be written to disk *before* the new value of X is written to disk
- (2) The log record $\langle \text{commit } T \rangle$ must be written to disk *after* all database elements changed by the transaction have been written to disk

Example

Step	Action	t	Mem A	Mem B	Disk A	Disk B	Mem Log
1							<start T>
2	Read(A,t)	8	8		8	8	
3	t:=t*2	16	8		8	8	
4	Write(A,t)	16	16		8	8	<T, A, 8>
5	Read(B,t)	8	16	8	8	8	
6	t:=t*2	16	16	8	8	8	
7	Write(B,t)	16	16	16	8	8	<T, B, 8>
8	Flush log						
9	Output(A)	16	16	16	16	8	
10	Output(B)	16	16	16	16	16	
11							<commit T>
12	Flush log						

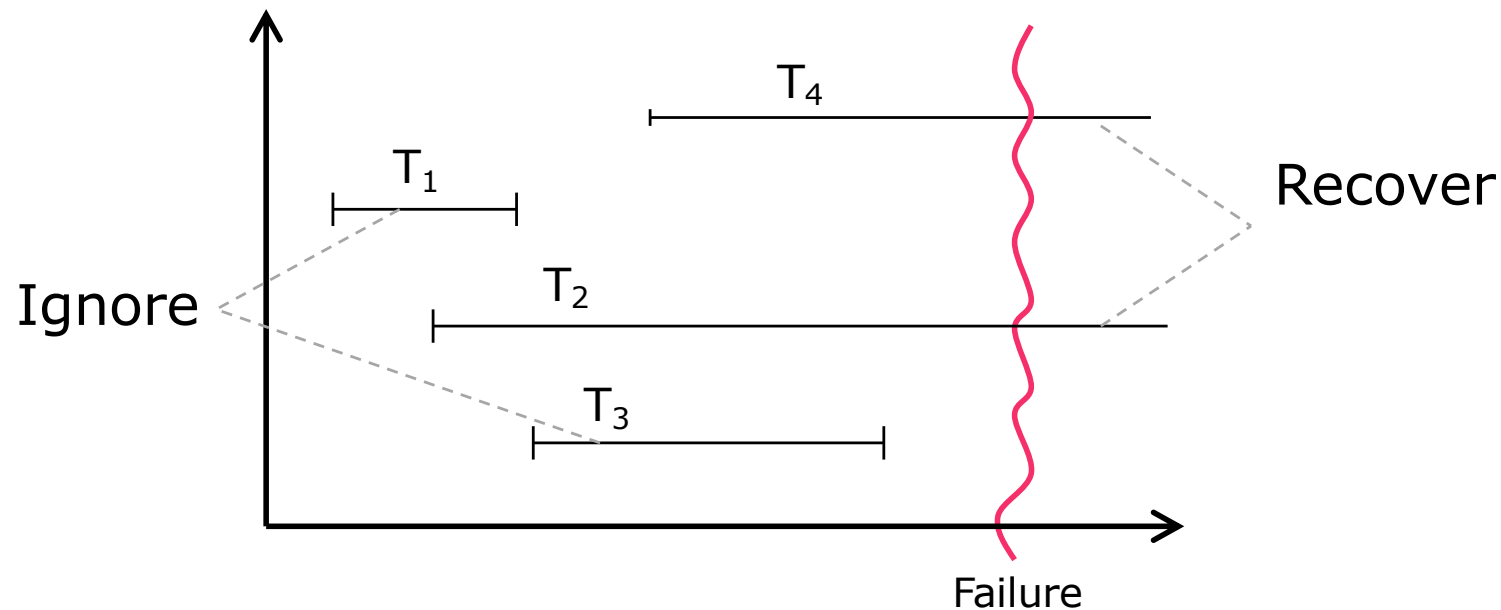
Undo-Logging

■ Recovery

- (1) Suppose that S is a set of incomplete transactions
 - We find $\langle \text{start } T \rangle$ but not $\langle \text{commit } T \rangle$ or $\langle \text{abort } T \rangle$ in the transaction log
- (2) Each $\langle T, X, v \rangle$ log record
 - If $T \in S$ then $\left\{ \begin{array}{l} - \text{Write}(X, v) \\ - \text{Output}(X) \end{array} \right.$
- (3) Each $T \in S$
 - Write a log record $\langle \text{abort } T \rangle$ on transaction log
 - Flush the log

Undo-Logging

- If crash happens
 - T_1 and T_3 are complete
 - T_2 and T_4 are incomplete



Example

Step	Action	t	Mem A	Mem B	Disk A	Disk B	Mem Log
1							<start T>
2	Read(A,t)	8	8		8	8	
3	t:=t*2	16	8		8	8	
4	Write(A,t)	16	16		8	8	<T, A, 8>
5	Read(B,t)	8	16	8	8	8	
6	t:=t*2	16	16	8	8	8	
7	Write(B,t)	16	16	16	8	8	<T, B, 8>
8	Flush log						<i>The changes to A/B do not reach disk</i>
9	Output(A)	16	16	16	16	8	
10	Output(B)	16	16	16	16	16	
11							<commit T>
12	Flush log						<i>Recover A=8 and B=8</i>
							<i>Do not undo the results of T</i>

Undo-Logging & Checkpoint

Checkpoint

<start T₁>
<T₁, A, 5>
<start T₂>
<T₂, B, 10>



- Since T₁ and T₂ are the active transactions, we have to wait
- When T₁ and T₂ complete, write <checkpoint> record on the log

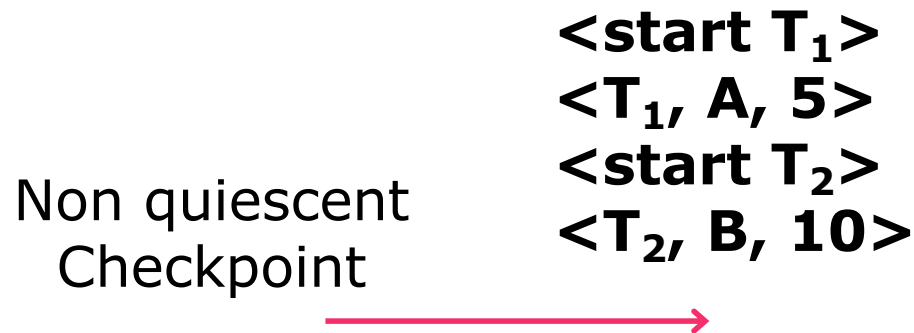
Example

<start T_1 >
< T_1 , A, 5>
<start T_2 >
< T_2 , B, 10>
< T_2 , C, 15>
< T_2 , D, 20>
<commit T_1 >
<commit T_2 >
<checkpoint>
<start T_3 >
< T_3 , E, 25>
< T_3 , F, 30>

scan

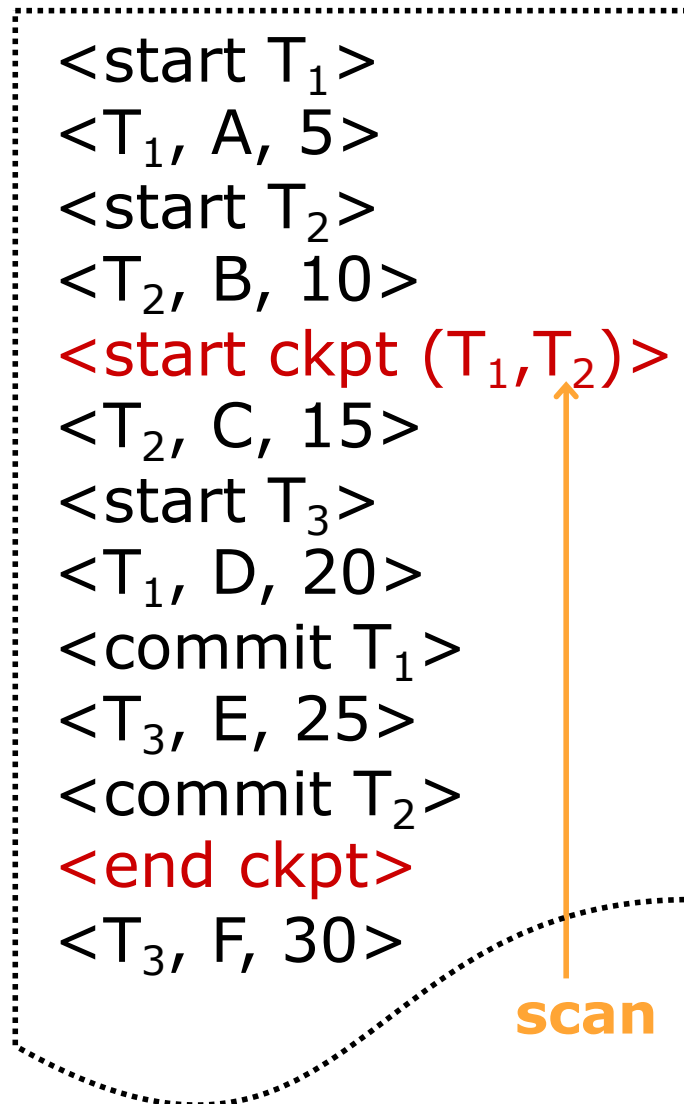
- < T_3 , F, 30>
 - T_3 is incomplete
 - Recover F=30
- < T_3 , E, 25>
 - Recover E=25
- Reach <checkpoint>
 - Stop

Undo-Logging & Checkpoint



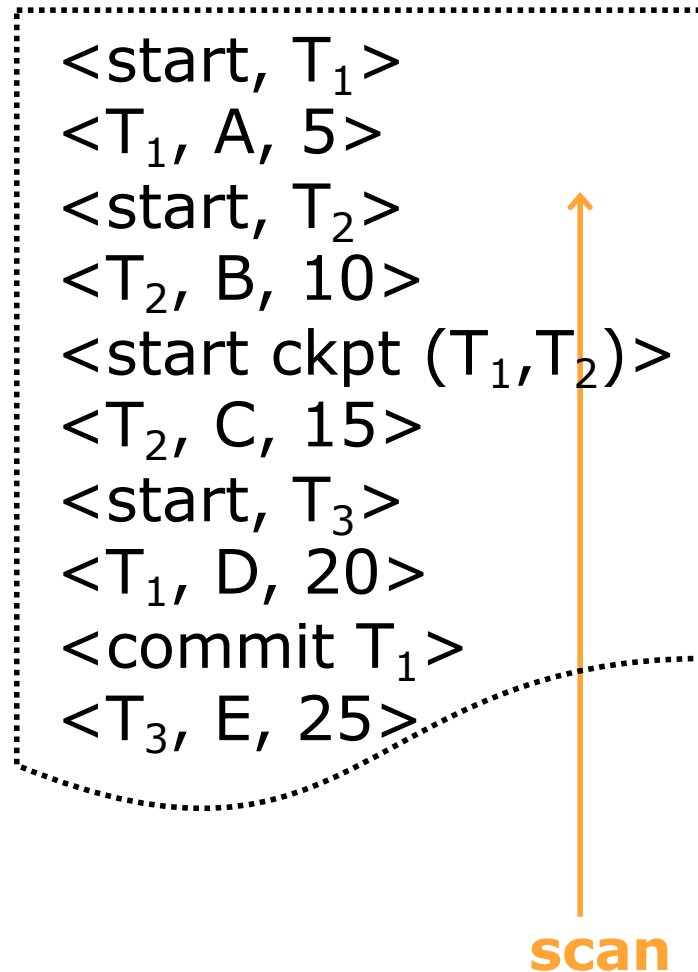
- Since T_1 and T_2 are the active transactions, we write a log record $\langle \text{start ckpt } (T_1, T_2) \rangle$
- While waiting for T_1 and T_2 complete, DBMS accepts another transactions
- When T_1 and T_2 complete, write $\langle \text{end ckpt} \rangle$ on the log

Example



- <T₃, F, 30>
 - T₃ is incomplete
 - Recover F=30
- <end ckpt>
 - T₁ and T₂ completed
- <T₃, E, 25>
 - Recover E=25
- Reach <start ckpt>
 - Stop

Example



- < T_3 , E, 25>
 - T_3 is incomplete
 - Recover E=25
- <commit T_1 >
 - T_1 completed
- < T_2 , C, 15>
 - Recover C=15
- <start ckpt(T_1, T_2)>
 - Only T_1 and T_2
- < T_2 , B, 10>
 - Recover B=10
- <start T_2 >
 - Stop

Content

- Introduction
- Transaction log
- Checkpoint
- Recovery method
 - Undo-logging (immediate modification)
 - **Redo-logging (deferred modification)**
 - Undo/Redo logging

Redo-Logging

■ Rules

- (0) Log record
 - $\langle T, X, v \rangle$
 - Only write the new value
- (1) **Before** modifying any DB element X on disk, all log records pertaining to this modification of X must appear on disk
 - $\langle T, X, v \rangle$ and $\langle \text{commit } T \rangle$
- (2) The $\langle \text{commit } T \rangle$ record must follow all the update log records

Example

Step	Action	t	Mem A	Mem B	Disk A	Disk B	Mem Log
1							<start T>
2	Read(A,t)	8	8		8	8	
3	t:=t*2	16	8		8	8	
4	Write(A,t)	16	16		8	8	<T, A, 16>
5	Read(B,t)	8	16	8	8	8	
6	t:=t*2	16	16	8	8	8	
7	Write(B,t)	16	16	16	8	8	<T, B, 16>
8							<commit T>
9	Flush log						
10	Output(A)	16	16	16	16	8	
11	Output(B)	16	16	16	16	16	

Redo-Logging

- In brief
 - (1) the log records indicating changed DB element
 - (2) the <commit> log record
 - (3) the changed DB element themselves

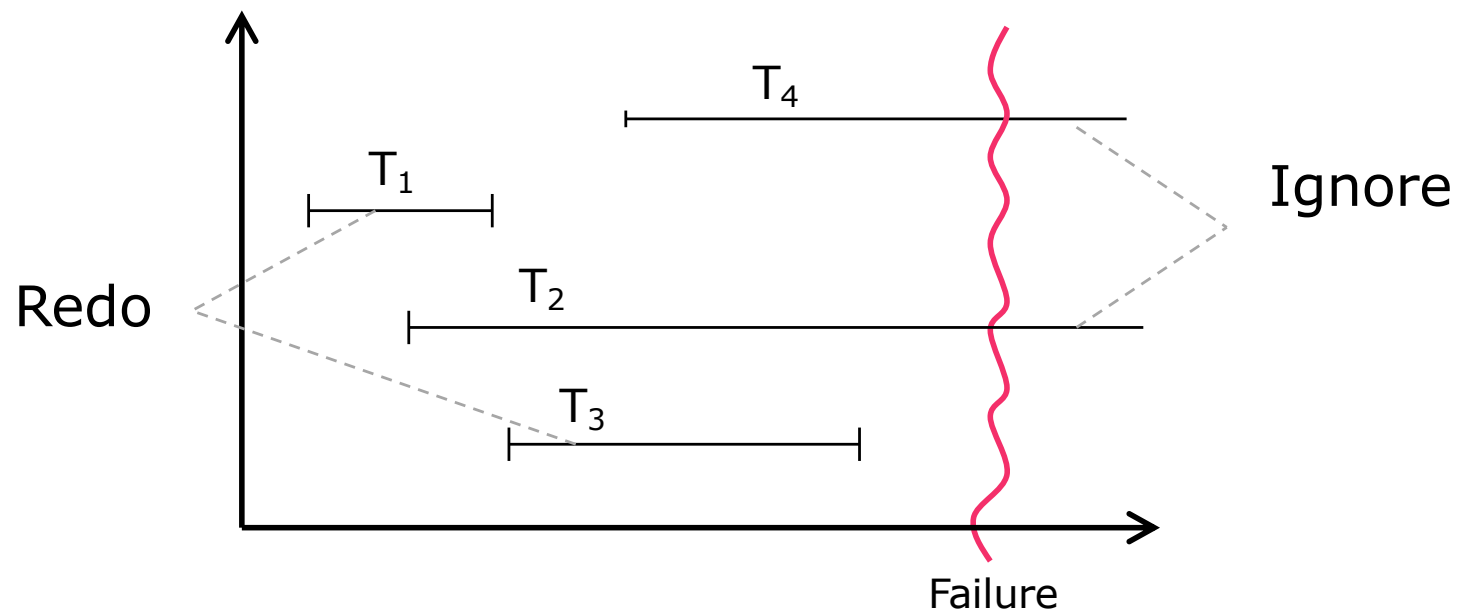
Redo-Logging

■ Recovery

- (1) Suppose that S is a set of complete transactions
 - Find $\langle \text{commit } T \rangle$ in the transaction log
- (2) For each log record $\langle T, X, v \rangle$
 - If $T \in S$ then $\left\{ \begin{array}{l} - \text{Write}(X, v) \\ - \text{Output}(X) \end{array} \right.$
- (3) For each $T \notin S$
 - Write an $\langle \text{abort } T \rangle$ record to the log
 - Flush the log

Redo-Logging

- If crash happens
 - T_1 and T_3 are complete
 - T_2 and T_4 are incomplete



Example

Step	Action	t	Mem A	Mem B	Disk A	Disk B	Mem Log
1							<start T>
2	Read(A,t)	8	8		8	8	
3	t:=t*2	16	8		8	8	
4	Write(A,t)	16	16		8	8	<T, A, 16>
5	Read(B,t)	8	16	8	8	8	
6	t:=t*2	16	16	8	8	8	
7	Write(B,t)	16	16	16	8	8	<T, B, 16>
8							<commit T>
9	Flush log						<i><commit T> has not reached disk, T is incomplete, no changes to A and B</i>
10	Output(A)	16	16	16	16	8	<i>Redo T, write A=16 and B=16</i>
11	Output(B)	16	16	16	16	16	<i>Redo T, write A=16 and B=16</i>

Redo-Logging & Checkpoint

■ Discussion

- The DB changes made by a committed transaction can be copied to disk much later than the time at which the transaction commits
- The time we decide to create a checkpoint
 - When???

Redo-Logging & Checkpoint

■ Discussion

- The nonquiescent checkpoint

- (1) Write a log record $\langle \text{start ckpt } (T_1, \dots, T_n) \rangle$ and flush the log
 - * $T_1, \dots, T_n$ are active (uncommitted) transactions
- (2) Write to disk all DB elements that were written to buffers but not yet to disk by transactions that had already committed when $\langle \text{start ckpt} \rangle$ record was written to the log
- (3) Write $\langle \text{end ckpt} \rangle$ record to the log and flush the log

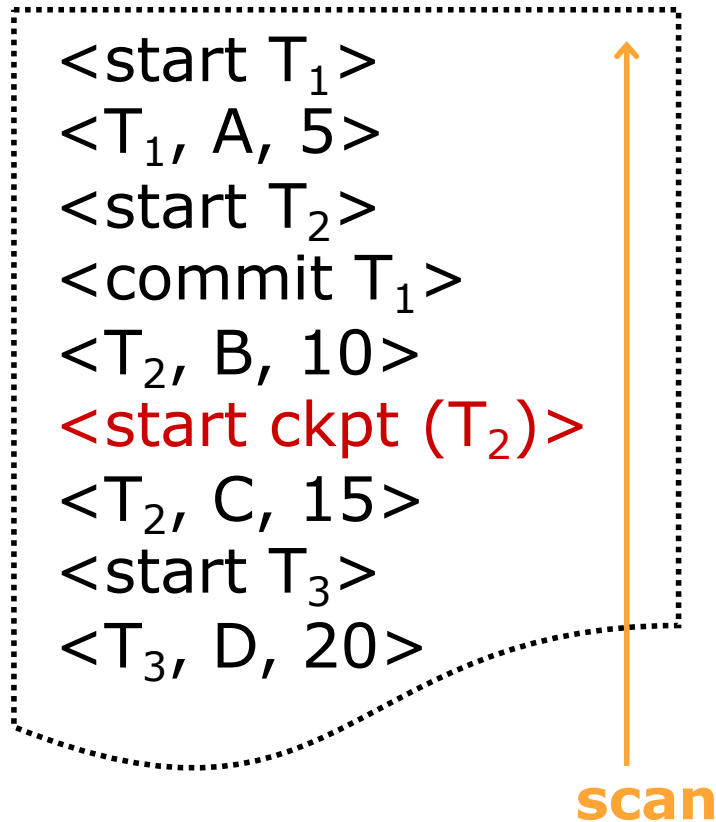
→ We can complete the checkpoint without waiting for the active transactions to commit or abort

Redo-Logging & Checkpoint

`<start T1>`
`<T1, A, 5>`
`<start T2>`
`<commit T1>`
`<T2, B, 10>`
`<start ckpt (T2)>`
`<T2, C, 15>`
`<start T3>`
`<T3, D, 20>`
`<end ckpt>`
`<commit T2>`
`<commit T3>`

- T₁ is complete before `<start ckpt>`
 - Value of A may be written to disk
 - If not, before `<end ckpt>`, A must be written to disk
- At `<start ckp>`
 - T₂ is active
 - T₃ starts
- After `<end ckpt>`
 - T₂ and T₃ are complete

Redo-Logging & Checkpoint



- T₂ and T₃ are incomplete
 - Do nothing for T₂ and T₃
- <commit T₁>
 - Redo T₁
 - Write A=5

Discussion

- Compare undo and redo logging
 - (1) During recovery
 - Undo
 - * Cancel the effect of incomplete transactions
 - * Ignore committed one
 - Redo
 - * Ignore incomplete transactions
 - * Repeat the changes made by committed transactions

Discussion

- Compare undo and redo logging
 - (2) Positions of the <commit> record
 - Undo
 - * Write changed DB elements to disk before the <commit> log record reaches disk
 - Redo
 - * The <commit> record appear on disk before any changed values reach disk

Discussion

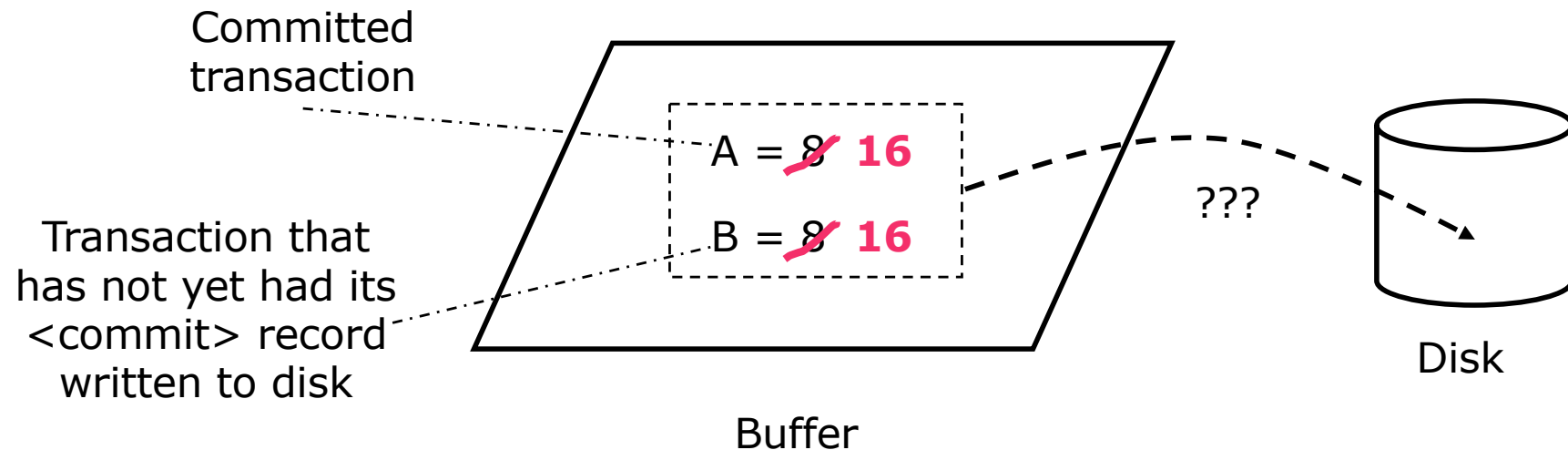
- Compare undo and redo logging
 - (3) Values of changed DB element
 - Undo
 - * The old value of changed DB elements are exactly what we need to recover
 - Redo
 - * The new values are needed

Discussion

- Undo logging (immediate modification)
 - Data have to be written to disk immediately after a transaction finishes
 - Increasing the number of disk I/Os that need to be performed
- Redo logging (deferred modification)
 - Keep all modified blocks in buffers until the transactions commits and the log records have been flushed
 - Increasing the average number of buffers required by transactions

Discussion

- How buffers are handled during a checkpoint
 - Undo and redo put contradictory requirements



- Required to copy the buffer to disk because of A
- But forbidden to do that because of B

Content

- Introduction
- Transaction log
- Checkpoint
- Recovery method
 - Undo-logging (immediate modification)
 - Redo-logging (deferred modification)
 - **Undo/Redo logging**

Undo/Redo - Logging

■ Rules

- (0) Log record
 - $\langle T, X, v, w \rangle$
 - v is the old value
 - w is the new value
- (1) **Before** modifying any DB element X on disk, the update record $\langle T, X, v, w \rangle$ appears on disk
- (2) The $\langle \text{commit } T \rangle$ log record can precede or follow any of the changes to the DB elements on disk

Example

Step	Action	t	Mem A	Mem B	Disk A	Disk B	Mem Log
1							<start T>
2	Read(A,t)	8	8		8	8	
3	t:=t*2	16	8		8	8	
4	Write(A,t)	16	16		8	8	<T, A, 8, 16>
5	Read(B,t)	8	16	8	8	8	
6	t:=t*2	16	16	8	8	8	
7	Write(B,t)	16	16	16	8	8	<T, B, 8, 16>
8	Flush log						
9	Output(A)	16	16	16	16	8	
10							<commit T>
11	Output(B)	16	16	16	16	16	

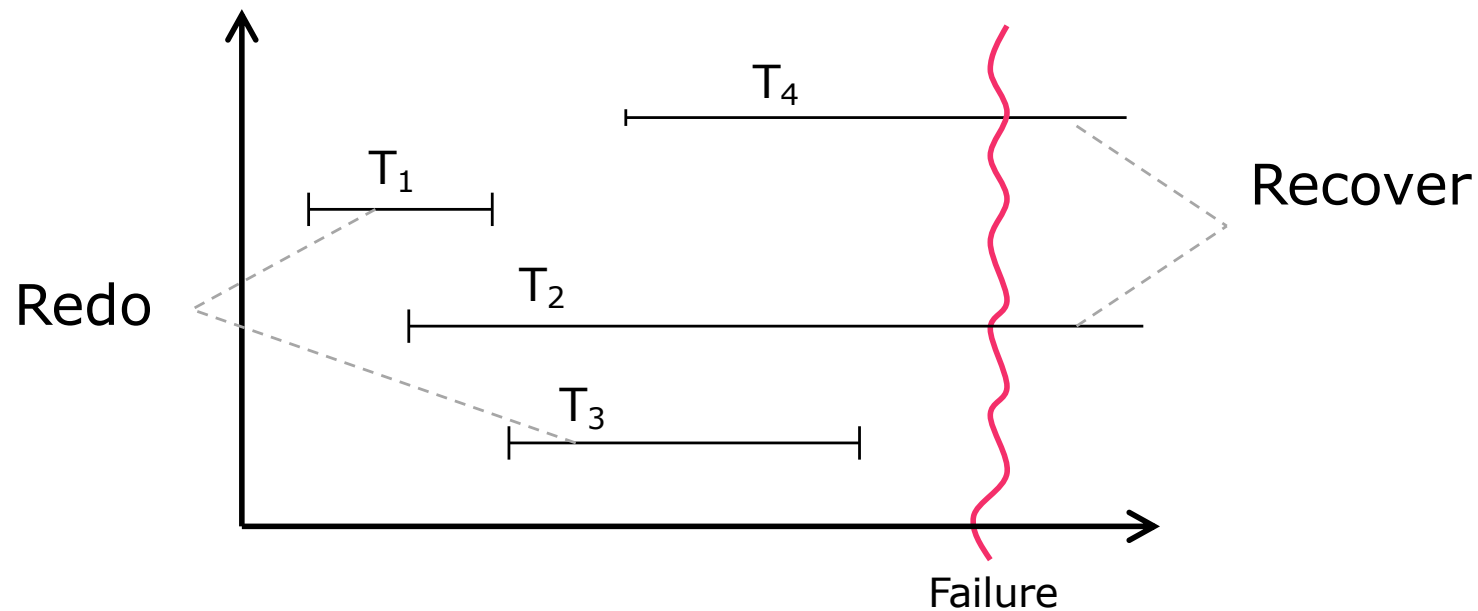
Undo/Redo - Logging

■ Recovery

- (1) Redo all the committed transactions
 - In the order earliest-first
- (2) Undo all incomplete transactions
 - In the order latest-first

Undo/Redo - Logging

- If crash happens
 - T_1 and T_3 are complete
 - T_2 and T_4 are incomplete



Example

Step	Action	t	Mem A	Mem B	Disk A	Disk B	Mem Log
1							<start T>
2	Read(A,t)	8	8		8	8	
3	t:=t*2	16	8		8	8	
4	Write(A,t)	16	16		8	8	<T, A, 8, 16>
5	Read(B,t)	8	16	8	8	8	
6	t:=t*2	16	16	8	8	8	
7	Write(B,t)	16	16	16	8	8	<T, B, 8, 16>
8	Flush log						
9	Output(A)	16	16	16	16	8	<i>T is incomplete, write A=8</i>
10							<commit T>
11	Output(B)	16	16	16	16	16	<i><commit T> is flushed to disk, write A=16 and B=16</i>

Undo/Redo-Logging & Chkpt

■ Checkpoint time

- (1) Write <start ckpt ($T_1, \dots, T_n$)> record to the log and flush the log
 - $T_1, \dots, T_n$ are all the active transactions
- (2) Write to disk all the buffers that contain one or more changed DB elements
 - We flush all buffers not just those written by committed transactions
- (3) Write <end ckpt> record to the log and flush the log

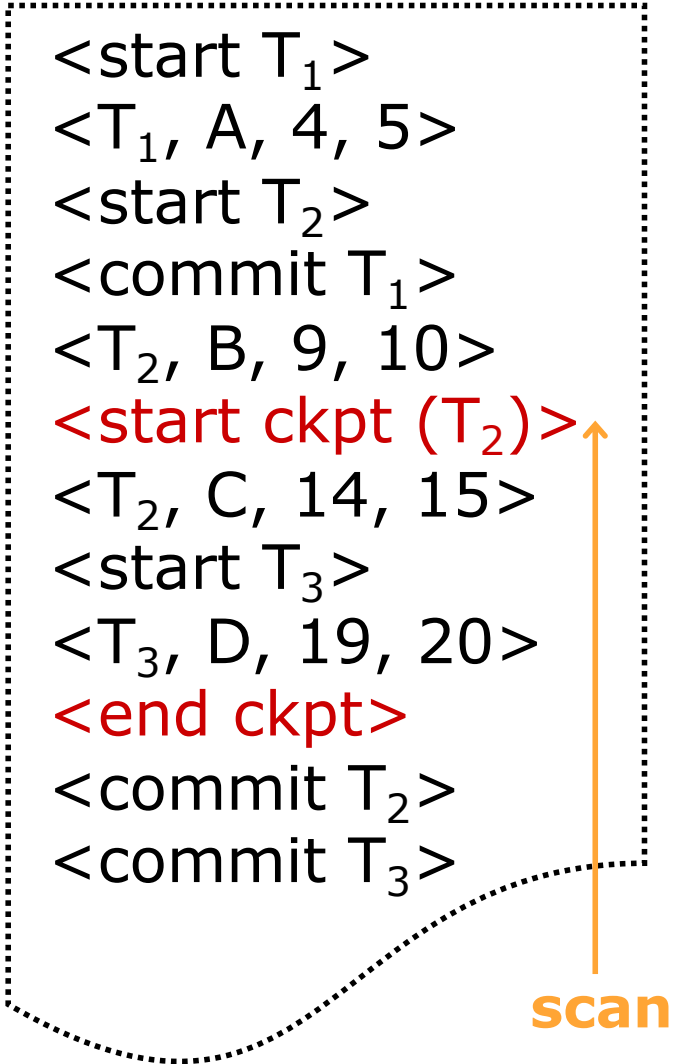
Example

`<start T1>`
`<T1, A, 4, 5>`
`<start T2>`
`<commit T1>`
`<T2, B, 9, 10>`
`<start ckpt (T2)>`
`<T2, C, 14, 15>`
`<start T3>`
`<T3, D, 19, 20>`
`<end ckpt>`
`<commit T2>`
`<commit T3>`

- T₁ is complete before `<start ckpt>`
 - Value of A may be written to disk
 - If not, before `<end ckpt>`, A must be written to disk
- Value of B=10 is written to disk

Example

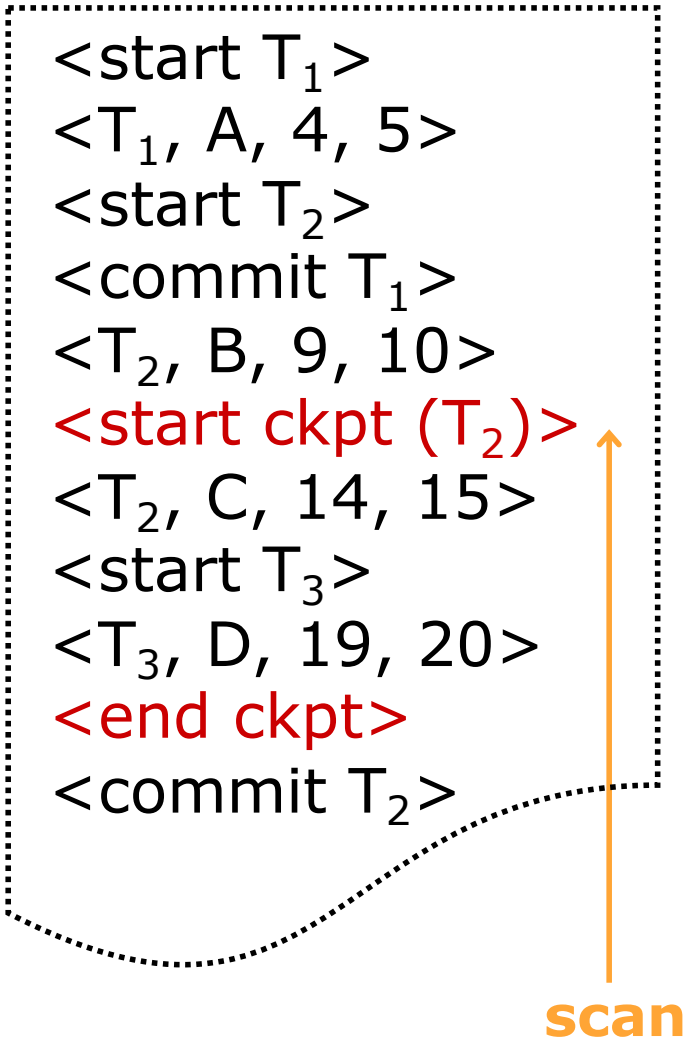
<start T_1 >
< T_1 , A, 4, 5>
<start T_2 >
<commit T_1 >
< T_2 , B, 9, 10>
<start ckpt (T_2)>
< T_2 , C, 14, 15>
<start T_3 >
< T_3 , D, 19, 20>
<end ckpt>
<commit T_2 >
<commit T_3 >



- Found <end ckpt>
 - Do nothing with T_1
- Examine T_2 and T_3
 - <commit T_2 >
 - Redo T_2
 - Write C=15
 - No need to write B
 - <commit T_3 >
 - Redo T_3
 - Write D=20

Example

<start T_1 >
< T_1 , A, 4, 5>
<start T_2 >
<commit T_1 >
< T_2 , B, 9, 10>
<start ckpt (T_2)>
< T_2 , C, 14, 15>
<start T_3 >
< T_3 , D, 19, 20>
<end ckpt>
<commit T_2 >



- Found <end ckpt>
 - Do nothing with T_1
- Examine T_2 and T_3
 - <commit T_2 >
 - Redo T_2
 - Write C=15
 - No need to write B
 - T_3 is incomplete
 - Recover D=19

Example

<start T_1 >
< T_1 , A, 4, 5>
<start T_2 >
<commit T_1 >
<start T_3 >
< T_2 , B, 9, 10>
< T_3 , E, 6, 7>
<start ckpt (T_2 , T_3)>
< T_2 , C, 14, 15>
< T_3 , D, 19, 20>
<end ckpt>
<commit T_2 >

- Found <end ckpt>
 - Do nothing with T_1
- Examine T_2 and T_3
 - <commit T_2 >
 - Redo T_2
 - Write C=15
 - No need to write B
 - T_3 is incomplete
 - Recover D=19 and E=6

